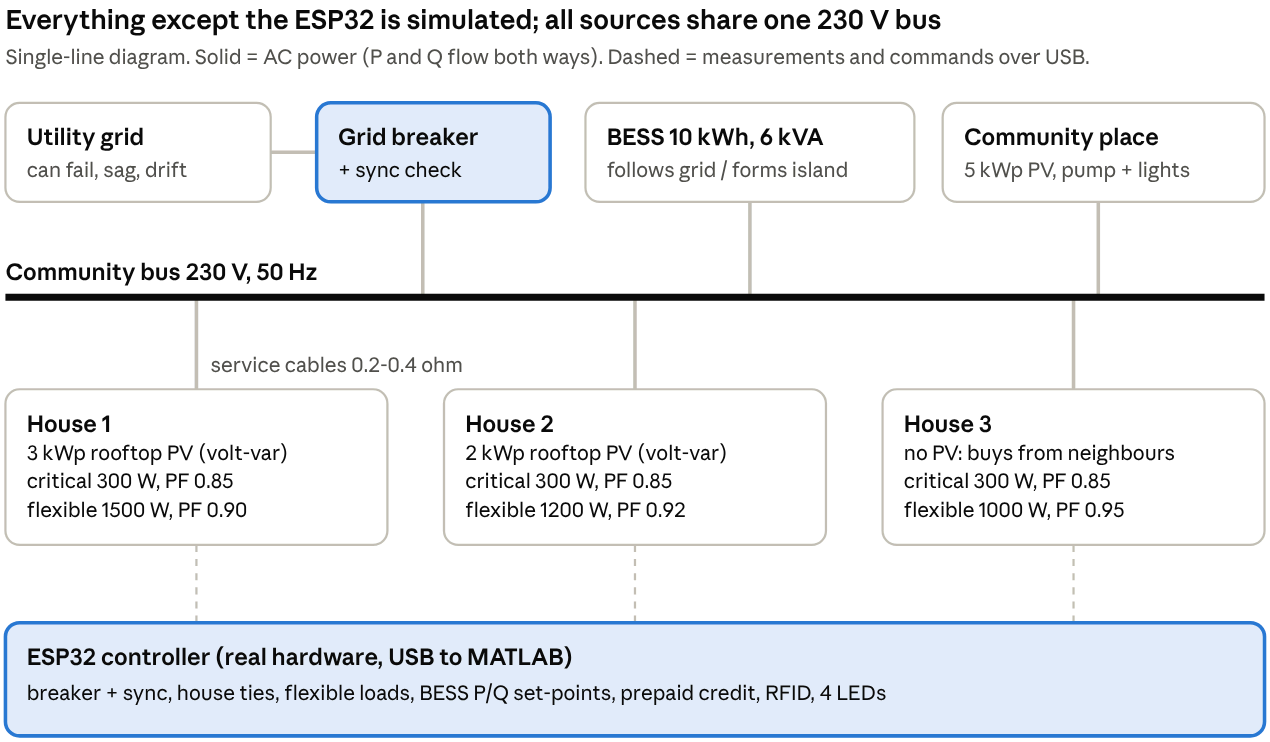
AC Microgrid Model: Learn It in One Sitting

Oct 7, 2026 · @Res

The picture

This is a separate model from the 12 V DC one, which stays unchanged in `mg_twin`. Here MATLAB simulates a real 230 V, 50 Hz neighbourhood: the grid, a community place with a shared battery (BESS) and a solar plant, and three houses on their own cables. The ESP32 is the only real hardware. It reads measurements over USB every second and decides what every switch does.



AC community microgrid · single-line diagram

All the numbers in the diagram live in one file, `mg_ac/params/ac_params.m`. Adding a fourth house is one copied row.

AC in five minutes

In DC, power is just V × I. In AC, voltage and current are sine waves, and the current can lag behind the voltage. That lag splits power into two parts.

Quantity	Symbol, unit	Plain meaning	In our model
Active power	P, watt (W)	Power that does real work (light, heat, pumping). This is what you pay for	Every load, PV, BESS, grid
Reactive power	Q, volt-amp reactive (var)	Power that sloshes back and forth to build the magnetic field of motors (fridge, pump, fan). Does no work but still needs current	Loads with power factor below 1
Apparent power	$S = \sqrt{(P^2 + Q^2)}$, VA	What the wires and inverters must actually carry	BESS inverter rated 6 kVA
Power factor	PF = P / S	1 = all useful; 0.8 = 20 % more current than needed	Fridge 0.85, pump 0.80
Frequency	f, Hz	How fast the sine wave turns; 50 Hz in India. Falls when generation < demand	Grid fixes it; in island the BESS sets it
Phase angle	θ, degrees	Where in its cycle a wave is. Two sources must match before connecting	dtheta between grid and microgrid

Why Q matters for a microgrid:

- **Current and losses:** Q makes current flow without delivering energy, so it heats cables and fills the inverter's rating.
- **Voltage drop:** current through a cable drops voltage roughly by R·P + X·Q (divided by V). So reactive power drags house voltages down.
- **Voltage rise:** exporting solar pushes voltage up. On a long cable that can go over limits; that is why rooftop inverters absorb a little Q when voltage is high (called volt-var).
- **What our BESS does:** when grid-connected, it supplies the community's Q itself. The grid then sees almost pure P, with a power factor of about 1. In the 24 h run the grid PF never dropped below 0.988.

The model is a **phasor (RMS) model**: it solves the steady AC state every second (amplitudes and angles), not the 50 Hz waveform itself. That is the standard tool for energy management and fast enough to run a day in 30 seconds.

Who sets voltage and frequency

An AC network needs exactly one "boss" that holds the voltage and frequency; every other source follows it. Which one is boss is the heart of islanding.

	Grid-connected	Islanded
Voltage and frequency set by	Utility grid (huge, stiff)	BESS inverter (**grid-forming**)
BESS behaves as	**Grid-following**: injects the P and Q the ESP32 asks for	Supplies whatever balance is needed, instantly
Power balance closed by	Grid import / export	BESS charge / discharge
Frequency	50 Hz (grid's)	50 Hz − 0.5 Hz × (BESS power / 5 kW)
Voltage	Grid voltage minus cable drops	230 V × (1 − 0.05 × BESS Q / 6 kVA)
PV inverters	Inject all available solar	Same, but cut back if frequency rises above 50.2 Hz

Droop is the trick in the last rows. A grid-forming inverter lets its frequency sag a little as it delivers more power, and its voltage sag a little as it delivers more vars. That way, frequency itself becomes a signal everyone can read:

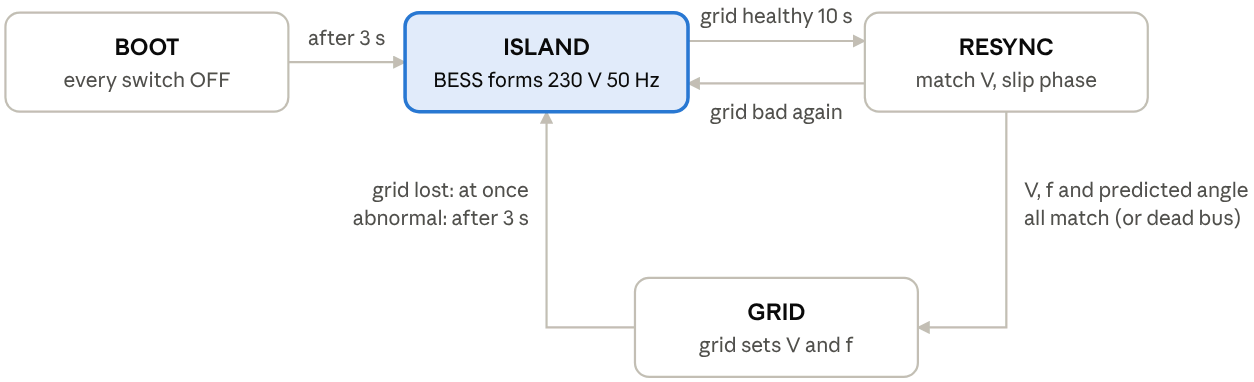
- **Frequency falling below 49.6 Hz** means the battery is near its power limit. The ESP32 sheds flexible loads.
- **Frequency rising above 50.2 Hz** means too much solar, and the battery can't take any more. The PV inverters curtail themselves (frequency-watt), with no communication needed.

The same principle runs real power systems: frequency is the shared heartbeat of supply versus demand.

The controller's four modes

The ESP32 runs a small state machine (named modes with fixed rules for moving between them). The key safety idea: the microgrid is live from the moment the BESS starts, so it never closes onto the grid blindly. It always goes through RESYNC.

The microgrid always joins the grid through synchronisation



ESP32 operating modes · 4 modes, 5 transitions

In GRID the breaker is closed (green LED). In ISLAND it is open (red LED). In RESYNC the red LED blinks while the BESS lines its waveform up with the grid. A healthy grid at power-up is joined within about 15 to 75 s; how long depends on where the phase gap starts.

Synchronising back to the grid

After an outage the microgrid has been running on its own clock. Its sine wave is no longer lined up with the grid's. Closing the breaker while the two waves are out of step is like engaging a gear at the wrong speed: a huge current spike. So the ESP32 closes **only when three things match**:

Condition	Our limit	Why
Voltage difference	⩽ 10 V	Unequal voltages push a current surge
Frequency difference	⩽ 0.2 Hz	Different speeds mean the angle keeps sliding

Condition	Our limit	Why
Phase angle at the moment of closing	⩽ 5°	Out-of-phase waves give the biggest surge

How it works in the model:

1. The grid has been healthy for 10 s, so the mode goes to **RESYNC** and the ESP32 raises the *sync request*.
2. The BESS then matches the grid voltage. It runs a tiny bit faster than the grid (6° per second of "slip"), so the angle difference sweeps slowly toward zero.
3. Each tick the ESP32 **predicts** where the angle will be when its command arrives (now + slip × 1 s). It closes only when that prediction is within ±5°. Real synchronisers do the same to allow for breaker closing time.
4. Breaker closed: the grid becomes boss again, and the BESS switches back to following P and Q set-points.

Exception, dead-bus closing: if the microgrid is dead (for example, the battery ran empty), there is nothing to synchronise with. The ESP32 closes straight away. Scenario S10 shows this recovery.

Verified: across all scenarios and Monte Carlo runs the worst closing angle is 5.0° (limit 10°).

Priorities and protection

One rule beats all others: **critical loads (fridge and lights in each house, the community pump and street lights) are never switched off by the controller**, not even at zero credit. Everything else is ranked. House 3 gives up first and House 1 last, set by the thresholds in the params file.

Trigger	Who acts	Action
Grid lost (below 0.5 pu)	ESP32, same tick	Open grid breaker, BESS forms the island (prevents back-feeding a dead line)
Grid abnormal (outside 0.85–1.10 pu or 49.5–50.5 Hz) for 3 s	ESP32	Island

Trigger	Who acts	Action
Island frequency below 49.6 Hz (BESS near its limit)	ESP32	Shed one flexible load per second, House 3 first; restore only if headroom allows
Island battery below 40 / 35 / 30 %	ESP32	Shed House 3 / 2 / 1 flexible load (8 % gap before restoring)
Island frequency above 50.2 Hz (battery full)	PV inverters themselves	Curtail solar (frequency-watt)
House voltage high or low	Rooftop inverters themselves	Absorb or inject vars (volt-var)
House current above 40 A for 2 s	ESP32	Trip that house, retry after 10 s, lock out after 2 trips
Prepaid balance ⩽ 0	ESP32	Flexible load off, critical stays on (debt logged)
3 bad RFID taps	ESP32	That reader locked for 60 s
Grid-connected, normal	ESP32	BESS covers community demand but never below 60 % (outage reserve), recharges from the grid at 1.5 kW when below it, and supplies all reactive power
E-stop pressed	ESP32	Everything open, white LED

The ESP32 shows its state on 4 LEDs: **green** = grid breaker closed, **red** = islanded (blinking while synchronising), **yellow** = some load shed, **white** = fault, lockout or E-stop.

Scenarios and verified results

All 10 scenarios pass their rules, 40 of 40 random Monte Carlo evenings pass, and the ESP32 C++ code gives byte-identical answers to the MATLAB controller over 195,000 ticks. Plots are in `mg_ac/results/figures`, one per scenario.

Scenario	What it shows	Key result
S1 normal day (24 h)	Grid-connected; BESS absorbs solar surplus, covers demand above 60 % SOC, supplies all vars	Grid PF ⩾ 0.988; voltages 0.984–1.012 pu; H1, H2 and community PV sell to H3
S2 grid outage 18:00–20:00	Instant islanding, priority shedding, sync back, reserve rebuilt	1 s transfer gap; closed at 1.2°; SOC 0.10 at the end of the outage, recharged after
S3 low battery outage	Battery-priority shedding	Shed order H3 → H2 → H1; critical loads 100 %
S4 island overload at peak	Under-frequency shedding before the BESS trips	Lowest 49.48 Hz, recovered; no trip
S5 island, sunny, battery full	Frequency-watt PV curtailment	Frequency up to 50.98 Hz (limit 51.5); no trip
S6 short circuit in House 3	Trip, retry, lockout	H3 locked out; H1, H2 unaffected
S7 voltage sag, then 49.2 Hz grid	Islands on abnormal grid, not just on outage	2 islandings, 2 clean reconnections
S8 cards and credit	Bad-card lockout, zero credit, top-up	Flexible load off at ₹0, critical stays on; top-up restores
S9 AI advisor	Unsafe or bad AI commands checked	Deep discharge clamped, bad price refused, island request ignored, 30 s fallback
S10 long outage	Battery empties → blackout → grid returns	Dead-bus close, critical loads restored

Bugs found and fixed while building (good viva material):

- **Boot closed onto a live island without syncing.** The closing angle was 71–86°. Fix: the controller always starts islanded and synchronises.
- **No recovery after a blackout.** A dead microgrid has nothing to sync with. Fix: dead-bus closing.
- **Battery too small for a 3 h evening outage.** It emptied. Fix: the reserve was raised to 60 %, and S10 now shows the empty-battery case honestly.

- **Battery stayed empty after an outage.** The rule only charged from solar. Fix: below the reserve it recharges from the grid at 1.5 kW.

What the AI/ML team gets and can send back

The microgrid gives the AI layer clean, timestamped data. It accepts one small command back, which the ESP32 checks before using. Full details are in `mg_ac/docs/data_contract.md`.

Data out:

- **Live:** a JSON telemetry line every 5 s.
- **1-second files:** CSVs for the normal-day and outage scenarios.
- **Training set:** 120 random days at 1-minute resolution (65 columns). It includes P, Q, voltage, current and frequency per house; community, BESS and grid values; prepaid balances; and mode.
- **Answer key:** label columns for outages, voltage sags, frozen load sensors, abnormal loads and RFID card abuse. Split train and test by `day_id`.

Command in: battery set-point, a mask of flexible loads to shed, and a price. The ESP32:

- clamps unsafe battery requests: never below 30 % SOC and never beyond the inverter limits;
- ignores battery requests in island, where physics rules;
- refuses prices outside ₹2–15/kWh;
- never lets AI touch a critical load;
- falls back to its own rules if the AI goes silent for 30 s.

Scenario S9 proves each of these.

This maps onto Zakir's three ideas:

- **Forecasting:** irradiance, PV and load columns.
- **Forecast-aware shedding:** acts through the battery set-point and the shed mask; the rule controller in the data is the baseline to beat.
- **Anomaly detection:** the label columns are the answer key.

30-minute code walk-through (folder `mg_ac`)

Open the files in this order. Each step answers one question.

1. **What numbers describe the system?** `params/ac_params.m` (5 min). Note the house table: copy a row to add a house.
2. **How does AC power flow?** `models/net_solve.m` (5 min). Each house current = conj(S / V); cable voltage drop = Z × I; repeat until the voltages stop changing. That is the whole AC solver.
3. **What happens in one second of the plant?** `models/plant_step.m` (8 min). Three cases:
   - **grid:** the BESS follows its set-points;
   - **island:** the BESS forms the voltage, with droop, solar curtailment and an overload timer;
   - **dead:** no supply.

   Read the sign conventions at the top.
4. **What does the ESP32 decide?** `controller/ac_controller_step.m` (8 min), top to bottom: grid health → mode → AI command → cards → under-frequency shedding → per-house protection, battery shedding and prepaid → BESS set-points. `firmware/src/controller.cpp` is the same logic in C++.
5. **How do they talk?** `docs/protocol.md` (2 min): one `M` line in, one `C` line out, every second.
6. **Did it work?** `results/results_summary.md` and `results/figures/S2_grid_outage.png` (2 min).

To try it live: in MATLAB, run the `addpath` line from the top of `run_all.m`, then `P = ac_params(); S = define_scenarios(P); R = run_scenario(S(2), P); plot_scenario(R, P, 'mine.png')`. Change one number in the params file (say BESS energy to 5 kWh), re-run S2, and see what breaks.

Viva questions and honest limits

Likely question	Short answer
Why open the breaker the moment the grid fails?	So the battery cannot feed a dead line and hurt a line worker (back-feeding), and so the BESS can take over as voltage source
Who keeps 50 Hz in island?	The BESS inverter (grid-forming, with droop). Frequency dips as it works harder, which tells the ESP32 to shed

Likely question	Short answer
Why does the BESS output vars?	Fridges and pumps need reactive power. Supplying it locally keeps the grid power factor near 1 and reduces current and voltage drop
What if solar exceeds demand in island and the battery is full?	Frequency rises past 50.2 Hz and the PV inverters curtail themselves (frequency-watt)
How do you reconnect safely?	Wait 10 s of healthy grid, match voltage, then slip slowly and close when the predicted phase gap is under 5°
What if the battery empties?	The inverter trips and the microgrid goes dark. When the grid returns the ESP32 does a dead-bus close (scenario S10). This is why the rules keep a 60 % reserve
Is the AI in control?	No. It advises; the ESP32 checks every command and falls back to its rules after 30 s of silence
How do you know the ESP32 code equals the model?	The SIL test feeds the same inputs to the C++ firmware and to MATLAB and compares every reply: 0 mismatches

Honest limits (say these before the jury asks):

- **Phasor model at 1 s steps.** No 50 Hz waveforms, harmonics or sub-cycle transients. Real islanding detection and breaker opening take milliseconds; in the model the transfer gap is one 1-second tick.
- **Over-current logic is not the real protection.** Real short-circuits are cleared by MCBs and fuses in milliseconds; the ESP32's over-current check here is logic only.
- **Single-phase, with typical and assumed values.** Cable impedances, BESS size and load profiles are synthetic or assumed, not measured.
- **Not yet run on the real board.** The firmware is compile-checked and proven equal to MATLAB, but it has not been built with the ESP32 toolchain or flashed.